

MEDALLION AZURE ADF DBT DATABRICKS

Modern Data Engineering with Azure Databricks + dbt + Unity Catalog + ADF

1. PROJECT OVERVIEW

This project implements a **production-grade Medallion Architecture** (Bronze → Silver → Gold) on Azure. It demonstrates the complete modern data platform lifecycle from raw source data to governed, business-ready analytical models.

Objective: Build a scalable, governed Lakehouse that ingests data from Azure SQL, processes it through Bronze/Silver/Gold layers using Unity Catalog for governance, dbt for transformations and snapshots, and Azure Data Factory for orchestration.

2. SYSTEM ARCHITECTURE

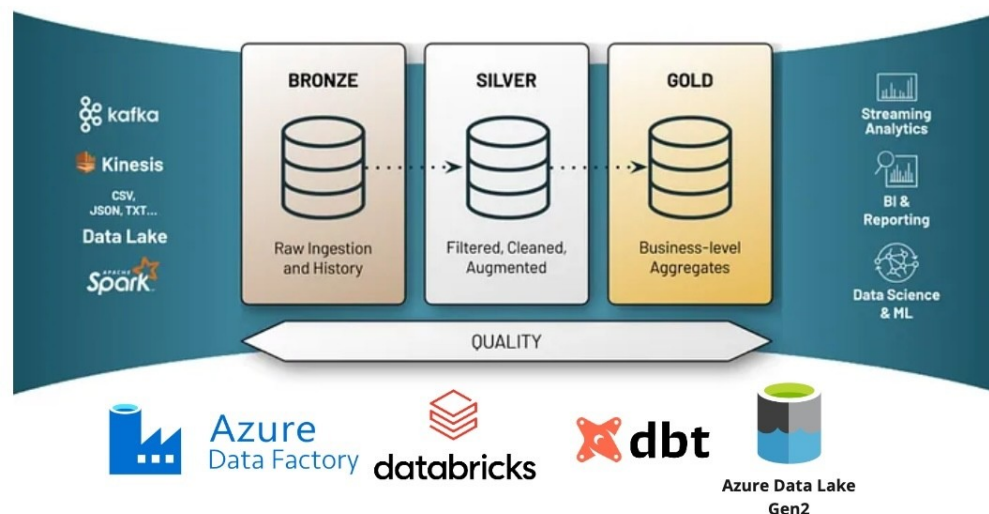


Figure 1: Medallion Architecture — Bronze (Raw) → Silver (SCD Snapshots) → Gold (Dimensional Models) with ADF, Databricks, dbt, and Unity Catalog

3. TECHNOLOGY STACK

Layer	Technology	Role
Source	Azure SQL Database (AdventureWorksLT)	Transactional source system
Orchestration	Azure Data Factory	Pipeline orchestration + Dynamic Notebook execution
Ingestion & Bronze	Azure Databricks + ADLS Gen2	Raw data landing + External tables in Unity Catalog
Silver (SCD)	dbt Snapshots	Slowly Changing Dimension Type 2 historical tracking
Gold (Modeling)	dbt Models (marts)	Dimensional modeling (dimensions + facts)

Governance	Unity Catalog	Centralized catalog, schemas, access control, lineage
Storage	Azure Data Lake Storage Gen2	Bronze, Silver, Gold containers + Unity Catalog metastore
Development	Databricks Notebooks + VS Code + dbt	Interactive development and debugging

4. PRE-REQUISITES — Azure Databricks Unity Catalog & Governance Setup

Before running any data pipeline, the following Unity Catalog infrastructure must be established:

Catalog > Create metastore >

Create metastore

Basics

Name* uc-metastore-central-india

Region* centralindia

Storage

ADLS Gen 2 path ⓘ <container_name>@<storage_account_na>
Optional location for storing managed tables data across all catalogs in the metastore. [Learn more](#)

Access Connector Id ⓘ /subscriptions/{sub-id}/resourceGroups/...

User assigned managed identity ID ⓘ /subscriptions/{sub-id}/resourceGroups/...

Assign to workspaces (optional)

Workspaces Assign workspaces

Cancel Create metastore

Figure 2: Unity Catalog Metastore created in Central India region

Create an Access Connector for Azure Databricks

Basics Tags **Managed Identity** Review + create

System assigned managed identity

Enable system assigned identity to grant the resource access to other existing resources.

Status ☐ Off ☒ On

User assigned managed identity

Add user assigned identities to grant the resource access to other existing resources.

+ Add Remove

Name	Resource group	Subscription
☐ medallion_managed_identity	medallion-spark-dbt-rg	5342102a-c05b-4637-b621-4ad9e...

Figure 3: Access Connector for Azure Databricks linked to Managed Identity

Create a new credential

A storage credential represents an authentication and authorization mechanism for accessing data stored on your cloud tenant. [Learn more](#)

☒ Storage Credential
 ☐ Service Credential

Credential Type*

Azure Managed Identity

Credential name*

adls-gen2-uc-credential

Access connector ID [Learn more](#)

User assigned managed identity ID (optional)

Comment

Managed Identity for medallion architecture project

> Advanced Options

Cancel Create

Figure 4: Storage Credential created for ADLS Gen2 access for external locations

- ****Key Setup Steps Performed:****

- create Azure Databricks resource
- Created Unity Catalog Metastore in Central India (<http://accounts.azuredatabricks.net/>)
- Created User-Assigned Managed Identity (medallion_managed_identity)
- Created Access Connector for Azure Databricks
- Created Storage Credential (adls-gen2-uc-credential) using Managed Identity
- Generated Personal Access Token for ADF → Databricks authentication

5. SOURCE LAYER — Azure SQL Database

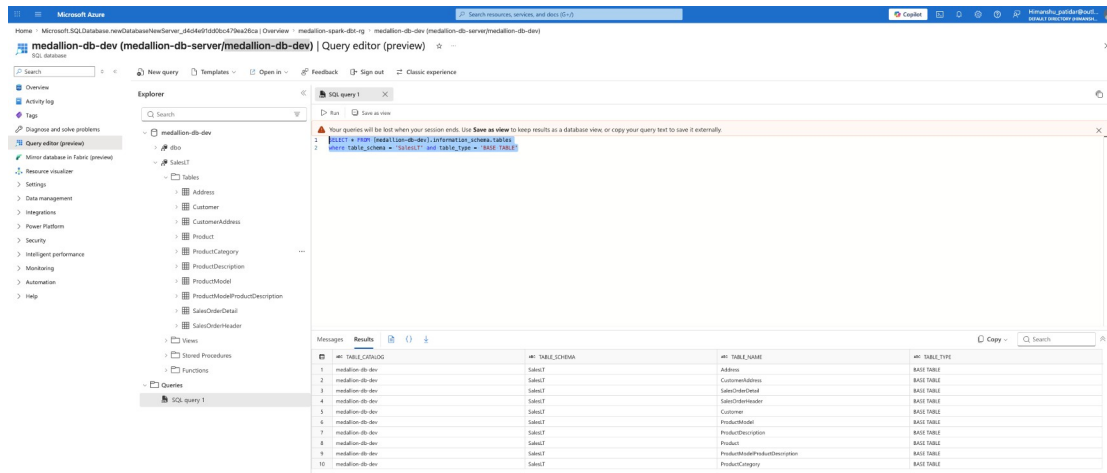


Figure 5: Azure SQL Database — SalesLT schema tables discovered via information_schema query

The project uses ****AdventureWorksLT**** sample database hosted on Azure SQL as the transactional source. Key tables include: Address, Customer, CustomerAddress, Product, ProductCategory, ProductDescription, ProductModel, ProductModelProductDescription, SalesOrderDetail, SalesOrderHeader.

6. ORCHESTRATION LAYER — Azure Data Factory Pipeline

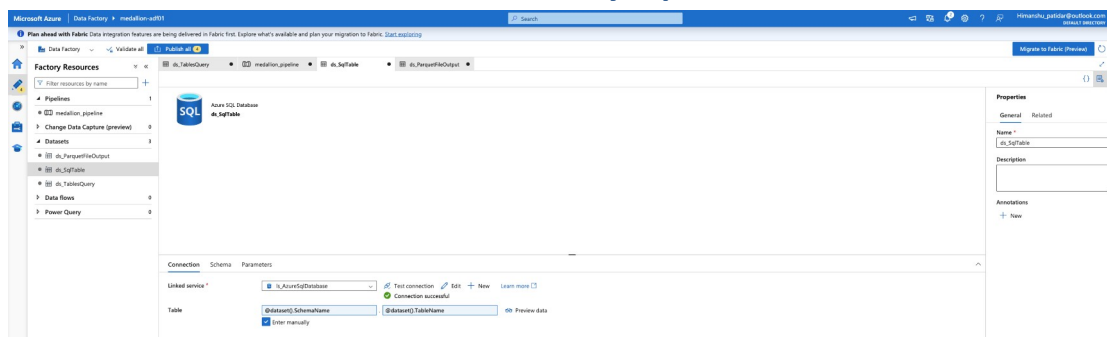


Figure 6: Azure Data Factory Pipeline datasets and linked services configuration

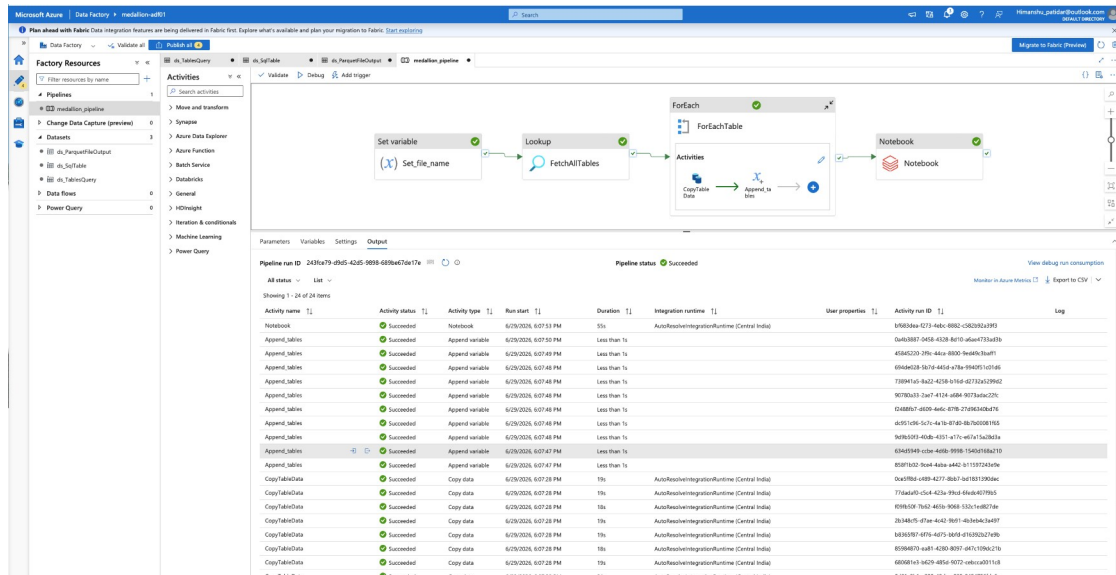


Figure 7: Azure Data Factory Pipeline 'medallion_pipeline' with Lookup + ForEach + Notebook + Copy activities, Successful pipeline execution showing all 24 activities completed with status Succeeded

- **Purpose:**** Orchestrate the entire data flow in a repeatable, scheduled, and monitorable manner. The ADF pipeline is the central orchestrator that triggers Bronze layer creation first.
- **Pipeline Activities (Correct Sequence):****
 - Lookup activity to fetch list of tables from source
 - ForEach + Append Variable to build dynamic JSON array of tables
 - Copy Data activities to load data into Bronze container in ADLS Gen2 (First output)
 - Databricks Notebook activity (at the end) to create external tables in Unity Catalog from Bronze files in container in ADLS Gen2

7. BRONZE LAYER — Raw Data Ingestion via ADF

****Purpose:**** Store data in its original/raw form with minimal transformation. This layer acts as the system of record.

- **Implementation Flow (Triggered by ADF Pipeline):****
 - Azure Data Factory Copy Activity extracts data from Azure SQL Database and writes it to Bronze container in ADLS Gen2 as Parquet files (First output of the pipeline).
 - In the same ADF pipeline, a Notebook Activity is triggered at the end.
 - The Notebook reads the list of tables dynamically and creates external tables in Unity Catalog under the 'saleslt' schema pointing to the Bronze Parquet files.

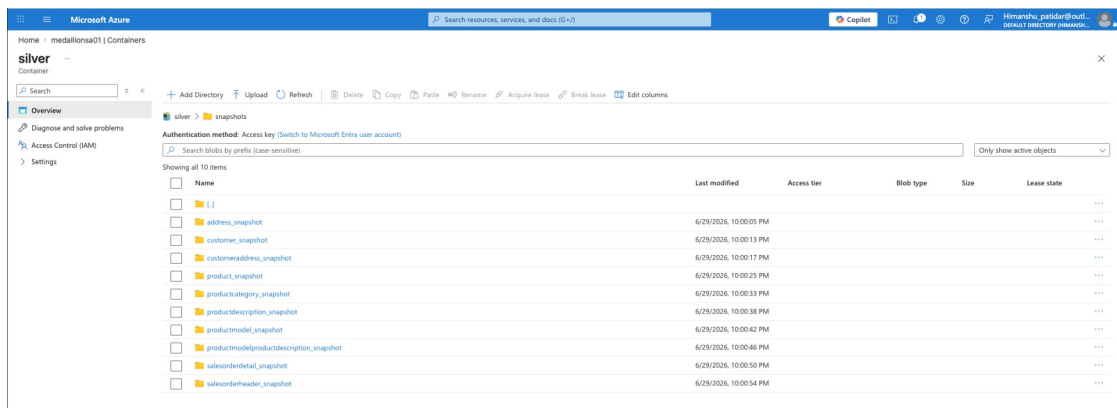


Figure 8: Bronze container in ADLS Gen2 with raw Parquet files from source tables (First output of ADF Pipeline)

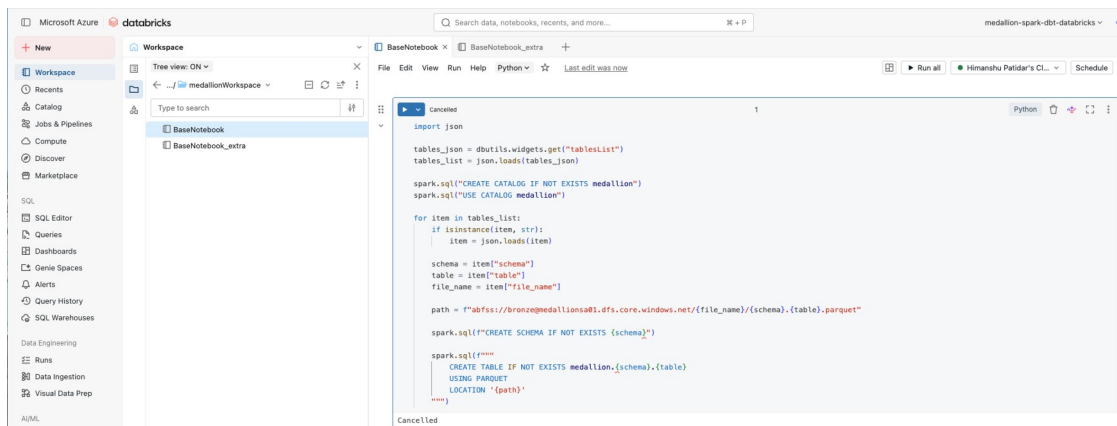


Figure 9: Databricks Notebook (BaseNotebook) that dynamically creates schemas and external tables in Unity Catalog from Bronze Parquet files (Executed after Bronze storage)

8. SILVER LAYER — dbt Snapshots (SCD Type 2)

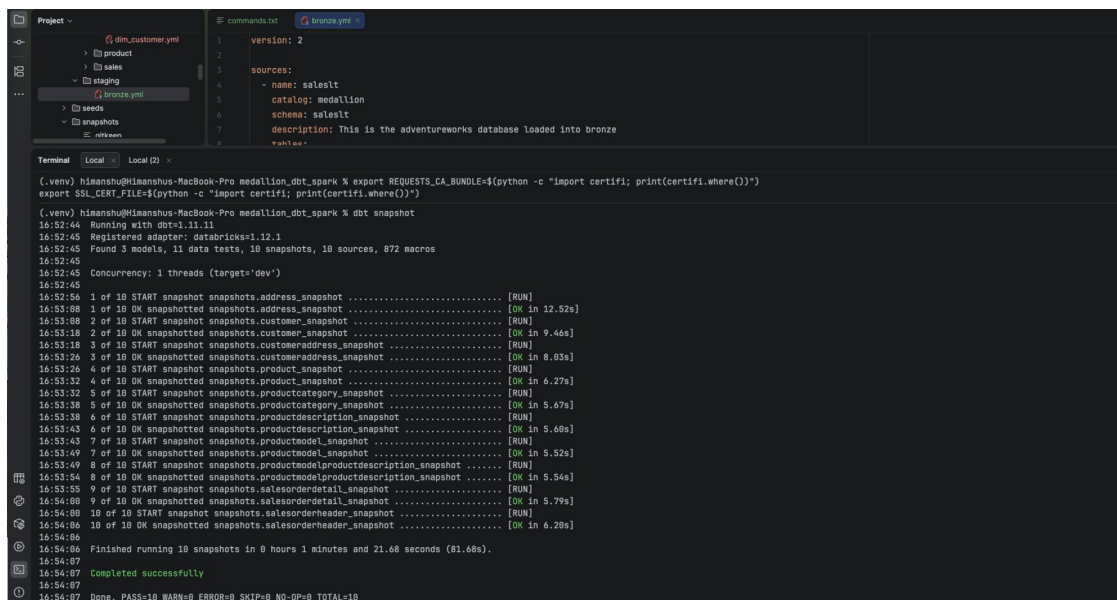


Figure 10: Successful dbt snapshot run — 10 snapshots executed (address, customer, product, salesorderdetail, etc.) in 81.68 seconds

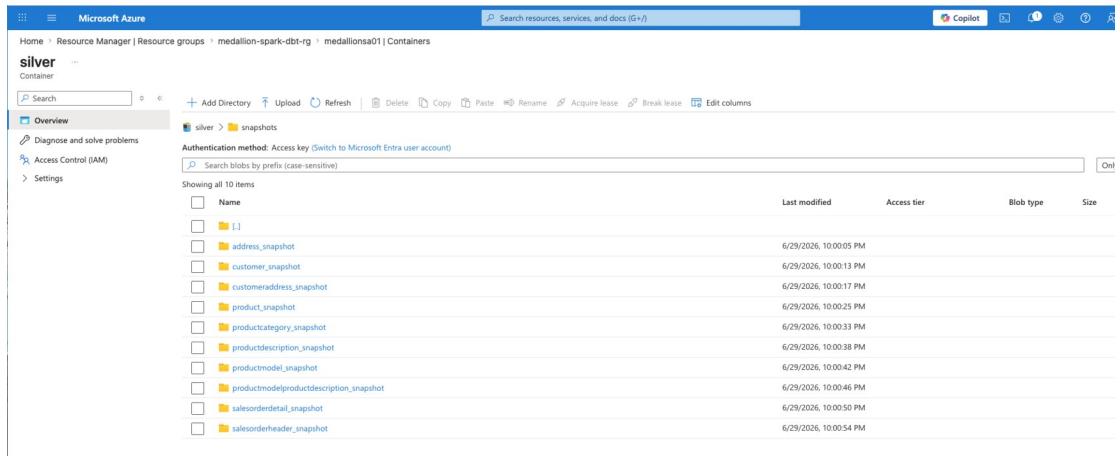


Figure 11: Silver container in ADLS Gen2 containing 10 snapshot folders with historical versions

Figure 12 shows the 'customeraddress_snapshot' table in the Unity Catalog. The table displays SCD columns: dbt_scd_id, dbt_valid_from, and dbt_valid_to. The table is structured as follows:

Sample	Customerid	Addressid	AddressType	dbt_scd_id	dbt_updated_at	dbt_valid_from	dbt_valid_to
1	29485	1086	Main Office	6542b90f15d1d88c40802297ada8741	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
2	29486	621	Main Office	b04eda057829c0d067321359eda1bc84	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
3	29489	1069	Main Office	50a45b9279eda9b79a218ac131606786	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
4	29490	887	Main Office	0fb693433ad173936300a40c11fe3f1a	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
5	29492	618	Main Office	1ed361537fec871f24051b918233346	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
6	29494	537	Main Office	3b2c31006207d08aea38c8045787bee0	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
7	29496	1072	Main Office	c700705870f6d76a9ee500340794182	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
8	29497	889	Main Office	6852736db05e16de679da0a8d835cd9	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
9	29499	527	Main Office	c220961911023a11cb39da1621e2bd57	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
10	29502	893	Main Office	07396685c7fa0e9bdf6aa749a9e0435	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
11	29503	32	Shipping	d73d1f9622b516d6da228008f8e06f4	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
12	29503	541	Main Office	9037070162192c7a1382c7ec413b89ba	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
13	29505	1083	Main Office	c55baa5902c0296973ab92dc8a455c87	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
14	29506	1082	Main Office	1cea76da5a834ccbe4de07589fad82	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
15	29508	619	Main Office	088fac1540a6758042bbdb839c412cf	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
16	29510	540	Main Office	dbda8bc5897a357acce25662c1e0d2f	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
17	29511	1046	Main Office	1be1d7e1b24c96c9546472a28f653e	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
18	29515	504	Main Office	01100856ea2129537f7a49ef18c498733	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
19	29517	794	Main Office	e7135518d9a116a2eed5ace1c006289	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
20	29521	1054	Main Office	3bbe250ab4a2256a9d758c0204bcb8a	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
21	29522	1040	Main Office	b16a12883abff52d1958520396de0f0	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
22	29523	593	Main Office	6b3c948ea0584a25da97cecc0f2022a	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
23	29524	599	Main Office	bb9018dc439c06c1de7b0ee5057b490a	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
24	29525	1049	Main Office	decf0ca0f40dfede0e2aadd7dfb8e	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
25	29527	512	Main Office	19f8b10d9e6a8d867d0600e6cdd2ad036	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
26	29528	788	Main Office	71140fb0e032a2c5f8aff33a01d402f	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
27	29530	495	Main Office	e207a96fa8b9867d0600e6cdd2ad036	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
28	29531	1061	Main Office	62a94572606c8bf9d596137c22a60ef3	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
29	29532	881	Main Office	d3acb78e3378ba2d6b64ee87546dfb9	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
30	29533	801	Main Office	56972a184af840ef52f2b1723a756549	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
31	29535	591	Main Office	01269b23798acda018955d6e3fc17d92	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
32	29536	786	Main Office	df6c6771fc65c538ac358788771d6ad	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
33	29539	480	Main Office	b0302e7fb885875197d3596f12c023ea	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
34	29541	467	Main Office	3dd0b462c30607b372712863c5642dfc	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
35	29544	519	Main Office	1843a3d7d326d3fc05b2db43dfa46d1	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
36	29545	11	Shipping	eb828989153506c0f740d9f9891b754	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
37	29545	834	Main Office	5504ba3d55a9e75b850090d0f868488	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
38	29546	635	Main Office	debd52146874d57e7426261a7b3f22	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null
39	29548	498	Main Office	3454ca5e974db84349c951246be2b49d3	2026-06-29T14:35:14.388+00:00	2026-06-29T14:35:14.388+00:00	null

Figure 12: customeraddress_snapshot table in Unity Catalog showing SCD columns (dbt_scd_id, dbt_valid_from, dbt_valid_to)

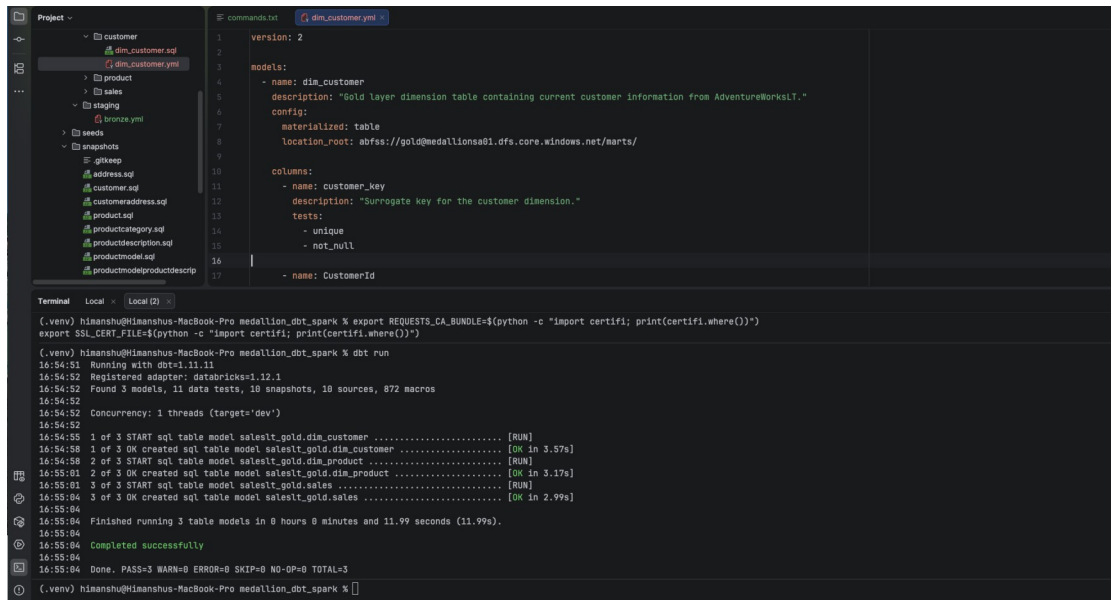
****Purpose:**** Capture historical changes using ****Slowly Changing Dimension (SCD) Type 2**** logic. This enables point-in-time analysis and maintains data lineage over time.

- **Implementation:****

- dbt project initialized and connected to Azure Databricks (using Databricks adapter).
- SSL certificate issues resolved by exporting REQUESTS_CA_BUNDLE and SSL_CERT_FILE.

- 10 dbt snapshots configured against Bronze tables in Unity Catalog.
- Snapshots create versioned records in the `snapshots` schema with valid_from/valid_to timestamps.
- Data is also written to Silver container in ADLS Gen2 for physical storage.

9. GOLD LAYER — Dimensional Modeling with dbt



The screenshot shows a VS Code editor with a project explorer on the left and a terminal at the bottom. The project explorer shows a directory structure with folders like 'customer', 'product', 'sales', 'staging', 'snapshots', and 'stgkeep'. The 'dim_customer.yml' file is selected in the 'customer' folder. The terminal shows the output of a dbt run command, including the model definition and the execution results.

```

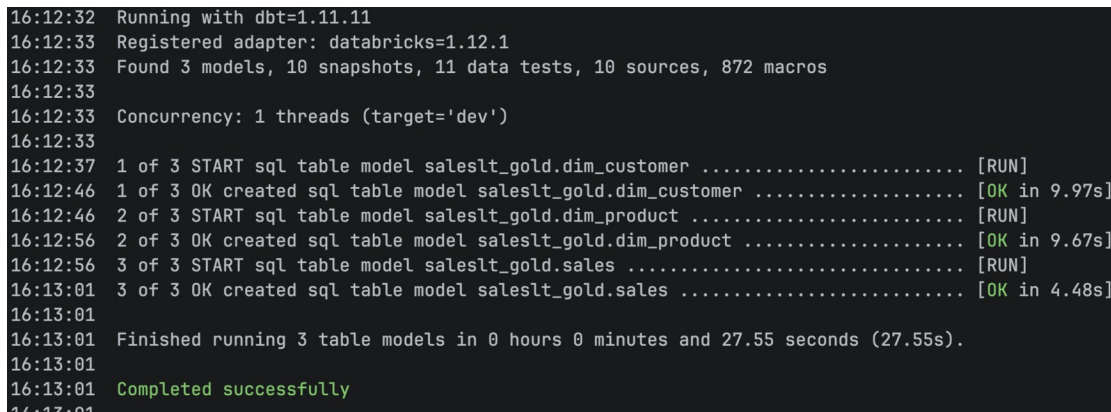
Project -
  customer
    dim_customer.sql
    dim_customer.yml
  product
  sales
  staging
  snapshots
  stgkeep
    address.sql
    customer.sql
    customeraddress.sql
    product.sql
    productcategory.sql
    productdescription.sql
    productmodel.sql
    productmodelproductdescrip

dim_customer.yml
1 version: 2
2
3 models:
4   - name: dim_customer
5     description: "Gold layer dimension table containing current customer information from AdventureWorksLT."
6     config:
7       materialized: table
8       location_root: abfss://gold@medallionsdb1.dfs.core.windows.net/marts/
9
10    columns:
11      - name: customer_key
12        description: "Surrogate key for the customer dimension."
13      tests:
14        - unique
15        - not_null
16
17      - name: CustomerId
  
```

```

(.venv) himanshu@Himanshu-MacBook-Pro medallion_dbt_spark % export REQUESTS_CA_BUNDLE=$(python -c "import certifi; print(certifi.where())")
export SSL_CERT_FILE=$(python -c "import certifi; print(certifi.where())")
(.venv) himanshu@Himanshu-MacBook-Pro medallion_dbt_spark % dbt run
16:54:51 Running with dbt=1.11.11
16:54:52 Registered adapter: databricks=1.12.1
16:54:52 Found 3 models, 11 data tests, 10 snapshots, 10 sources, 872 macros
16:54:52 Concurrency: 1 threads (target='dev')
16:54:52
16:54:52 1 of 3 START sql table model saleslt_gold.dim_customer ..... [RUN]
16:54:58 1 of 3 OK created sql table model saleslt_gold.dim_customer ..... [OK in 3.57s]
16:54:58 2 of 3 START sql table model saleslt_gold.dim_product ..... [RUN]
16:55:01 2 of 3 OK created sql table model saleslt_gold.dim_product ..... [OK in 3.17s]
16:55:01 3 of 3 START sql table model saleslt_gold.sales ..... [RUN]
16:55:04 3 of 3 OK created sql table model saleslt_gold.sales ..... [OK in 2.99s]
16:55:04
16:55:04 Finished running 3 table models in 0 hours 0 minutes and 11.99 seconds (11.99s).
16:55:04 Completed successfully
16:55:04 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
(.venv) himanshu@Himanshu-MacBook-Pro medallion_dbt_spark %
  
```

Figure 13: dbt model for dim_customer (Gold layer dimension with surrogate key and data quality tests)



The screenshot shows a terminal window with the output of a dbt run command. The output indicates that 3 models (dim_customer, dim_product, sales) were successfully created in 11.99 seconds.

```

16:12:32 Running with dbt=1.11.11
16:12:33 Registered adapter: databricks=1.12.1
16:12:33 Found 3 models, 10 snapshots, 11 data tests, 10 sources, 872 macros
16:12:33
16:12:33 Concurrency: 1 threads (target='dev')
16:12:33
16:12:37 1 of 3 START sql table model saleslt_gold.dim_customer ..... [RUN]
16:12:46 1 of 3 OK created sql table model saleslt_gold.dim_customer ..... [OK in 9.97s]
16:12:46 2 of 3 START sql table model saleslt_gold.dim_product ..... [RUN]
16:12:56 2 of 3 OK created sql table model saleslt_gold.dim_product ..... [OK in 9.67s]
16:12:56 3 of 3 START sql table model saleslt_gold.sales ..... [RUN]
16:13:01 3 of 3 OK created sql table model saleslt_gold.sales ..... [OK in 4.48s]
16:13:01
16:13:01 Finished running 3 table models in 0 hours 0 minutes and 27.55 seconds (27.55s).
16:13:01
16:13:01 Completed successfully
16:13:01
  
```

Figure 14: Successful dbt run creating 3 Gold models (dim_customer, dim_product, sales) in 11.99 seconds

Figure 15 shows the Gold layer 'sales' fact table in the Unity Catalog under the saleslt_gold schema. The table is a fact table with columns: SalesOrderID, OrderDate, DueDate, ShipDate, Status, OnlineOrderFlag, CustomerID, and SalesOrder. The table contains 27 rows of data.

Sample	SalesOrderID	OrderDate	DueDate	ShipDate	Status	OnlineOrderFlag	CustomerID	SalesOrder
1	71774	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	29847	
2	71776	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	29847	
3	71776	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30072	
4	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
5	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
6	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
7	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
8	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
9	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
10	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
11	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
12	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
13	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
14	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
15	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
16	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
17	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
18	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
19	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
20	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
21	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
22	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
23	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
24	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
25	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
26	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	
27	71780	2008-06-01T00:00:00.000+00:00	2008-06-13T00:00:00.000+00:00	2008-06-08T00:00:00.000+00:00	5	false	30113	

Figure 15: Gold layer 'sales' fact table in Unity Catalog under saleslt_gold schema

Figure 16 shows the Gold container in ADLS Gen2 with the marts/sales folder containing Delta files. The container contains a 'marts' folder, which contains three Delta files: dim_customer, dim_product, and sales.

Name	Last modified	Access tier	Blob type	Size	Lease state
dim_customer	6/29/2026, 9:42:40 PM				...
dim_product	6/29/2026, 9:42:48 PM				...
sales	6/29/2026, 9:42:58 PM				...

Figure 16: Gold container in ADLS Gen2 with marts/sales folder containing Delta files

Purpose: Transform cleaned and historical data into business-ready dimensional models optimized for analytics and BI.

Implementation:

- dbt models created in 'marts' folder reading from Silver snapshots.
- Dimension tables: dim_customer, dim_product (with surrogate keys and SCD attributes).
- Fact table: sales (fact table with foreign keys to dimensions).
- All models materialized as tables in the 'saleslt_gold' schema in Unity Catalog.
- Data also written to Gold container in ADLS Gen2 under marts/ folder.

10. DBT DOCUMENTATION & LINEAGE

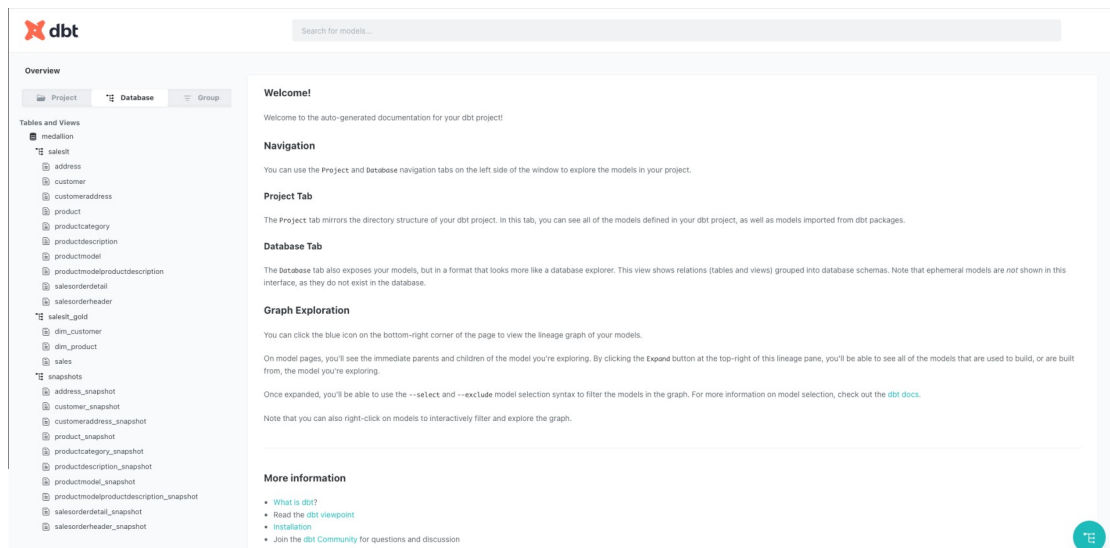


Figure 17: dbt Documentation site generated for the project

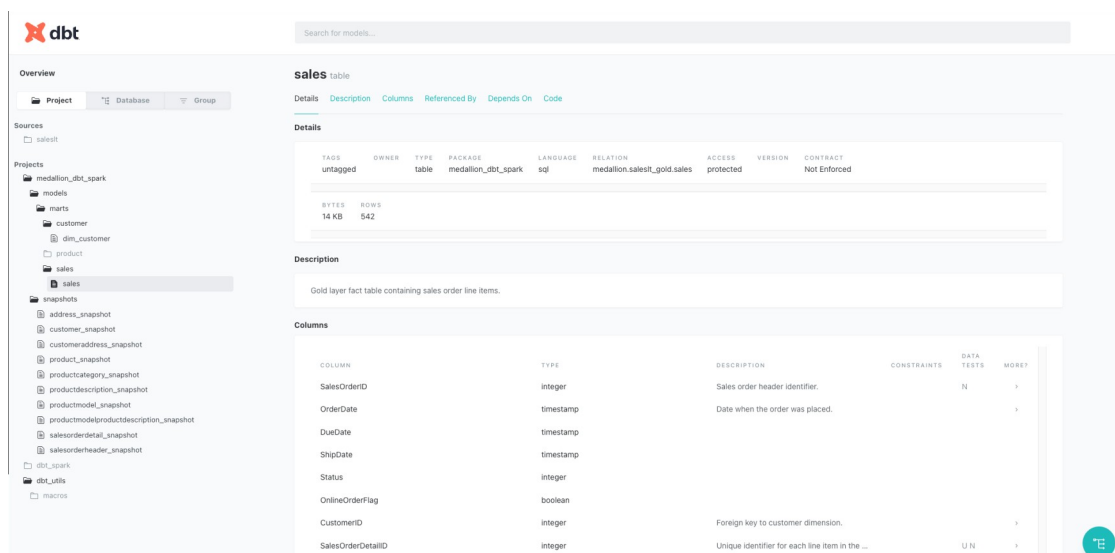


Figure 18: dbt Documentation showing models, snapshots, and lineage graph

After completing Silver and Gold layers, dbt documentation was generated to provide:

- Auto-generated documentation for all models and snapshots
- Lineage graph showing dependencies between models
- Column-level descriptions and data quality test results
- Interactive exploration of the data transformation flow

11. END-TO-END DATA FLOW

- ****Step 1: Pre-requisites (Unity Catalog Setup)****
 - Create Metastore, Access Connector, Managed Identity, Storage Credential, and Catalog with schemas.

- ****Step 2: Source Data Discovery****
 - Query Azure SQL Database to identify tables in SalesLT schema.
- ****Step 3: Orchestration via ADF Pipeline****
 - ADF Pipeline is triggered and orchestrates the entire flow.
- ****Step 4: Bronze Layer (ADF Pipeline Output)****
 - ADF Pipeline runs Copy Activity → Data loaded to Bronze container in ADLS Gen2 as Parquet (First output).
 - Notebook Activity (triggered at end of same pipeline) creates external tables in Unity Catalog under saleslt schema from Bronze files.
- ****Step 5: Silver Layer (dbt Snapshots)****
 - dbt initialized and connected to Azure Databricks.
 - dbt snapshot command executed → 10 snapshots created in snapshots schema (SCD Type 2).
 - Snapshot data also stored in Silver container in ADLS Gen2.
- ****Step 6: Gold Layer (dbt Models)****
 - dbt run command executed on marts → 3 models created (dim_customer, dim_product, sales).
 - Gold tables created in saleslt_gold schema in Unity Catalog.
 - Data also written to Gold container in ADLS Gen2 under marts/.
- ****Step 7: Documentation & Verification****
 - dbt docs generate executed to create documentation site with lineage.
 - Final verification performed in Unity Catalog Explorer (all schemas visible).

12. KEY IMPLEMENTATION CHALLENGES & SOLUTIONS

- ****Challenge 1: Unity Catalog Access Setup****
 - ****Solution:**** Proper sequence — Metastore → Managed Identity → Access Connector → Storage Credential with correct RBAC (Storage Blob Data Contributor).
- ****Challenge 2 dbt + Databricks SSL Issues on macOS****
 - ****Solution:**** Export CA bundle and SSL certificate using certifi before running dbt:


```
export REQUESTS_CA_BUNDLE=$(python -c "import certifi; print(certifi.where())")
export SSL_CERT_FILE=$(python -c "import certifi; print(certifi.where())")
```
- ****Challenge 3: Dynamic Multi-Table Processing in ADF****
 - ****Solution:**** Use Lookup + ForEach + Append Variable (JSON array) → Pass to Databricks Notebook for dynamic external table creation.
- ****Challenge 4: Snapshot vs Model Strategy****

- **Solution:** Snapshots only for SCD Type 2 dimensions. Regular models for facts and current-state dimensions.

13. KEY ACHIEVEMENTS

- Full Medallion Architecture implemented end-to-end on Azure with correct pipeline flow
- Unity Catalog properly configured with Metastore, Credentials, Access Connector, and Managed Identity
- dbt snapshots successfully implementing SCD Type 2 (10 snapshots executed)
- Clean Gold layer dimensional models with surrogate keys and data quality tests
- Robust ADF orchestration with dynamic Notebook integration for Bronze layer
- Complete governance, documentation, and lineage using Unity Catalog + dbt docs

14. SKILLS DEMONSTRATED

- Medallion / Lakehouse Architecture design and implementation
- Unity Catalog governance, metastore setup, and access management
- dbt for snapshots (SCD Type 2), modeling, testing, and documentation
- Azure Databricks + Spark for large-scale data processing
- Azure Data Factory for complex orchestration and dynamic pipelines
- Infrastructure as Code mindset for Unity Catalog and identity setup
- End-to-end data platform development from source to analytics consumption
- Troubleshooting SSL, authentication, and dynamic pipeline issues in hybrid environments

15. FINAL VERIFICATION — Unity Catalog Explorer

After completing the entire pipeline (Bronze → Silver → Gold), the final state of Unity Catalog was verified:

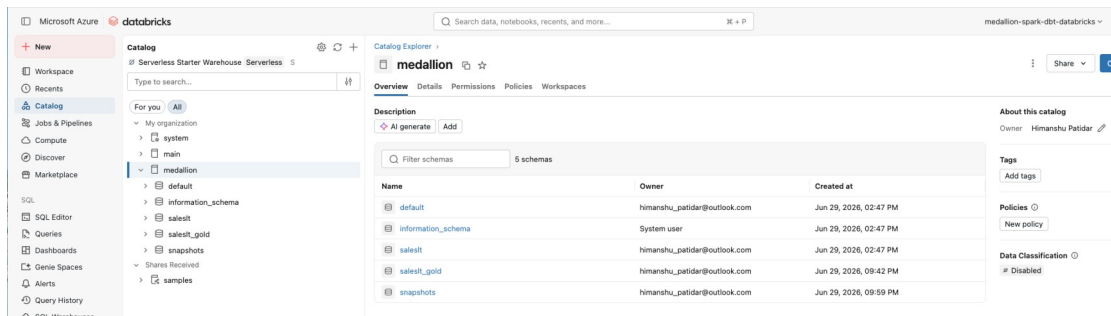


Figure 19: Unity Catalog 'medallion' with all schemas (saleslt, saleslt_gold, snapshots) — Final output after running all pipeline steps

This confirms that all layers were successfully created and governed under Unity Catalog.